



- 1. Purpose / Goal**
- 2. Introduction to Java 17**
- 3. key New features**
- 4. Frequently asked Interview Questions**

Agenda

⑩ Introduction to Java 17

- Java 17 Overview
- Key Highlights
- Importance of Upgrading

• Key New Features in Java 17

- Pattern Matching for switch (Preview)
- Sealed Classes
- Enhanced Pseudo-Random Number Generators
- New Applet API Removal
- Context-Specific Deserialization Filters
- Foreign Function & Memory API (Incubator)
- Vector API (Second Incubator)
- Strong Encapsulation of JDK Internals by Default
- Deprecate and Disable Biased Locking
- macOS/AArch64 Port



Introduction to Java 17

■ Java 17 Overview:

- Released in September 2021
- Long-Term Support (LTS) version, supported until 2029
- Successor to Java 11 LTS, following the 6-month release cadence
- Part of Oracle's commitment to a predictable release schedule

■ Key Highlights:

- Enhanced performance and stability
- Significant improvements in garbage collection and memory management
- Introduction of new language features and APIs

■ Importance of Upgrading:

- Staying updated with the latest security patches and bug fixes
- Leveraging new features for more efficient and readable code
- Ensuring compatibility with modern libraries and frameworks

Benefits of Upgrading to Java 17

Long-Term Support

Java 17 is an LTS release, ensuring extended support and updates, making it a stable and reliable choice for production environments

Improved Security

Strong encapsulation of JDK internals and context-specific deserialization filters enhance application security by reducing vulnerabilities

Developer Productivity

New APIs and language enhancements, such as the Vector API and Foreign Function & Memory API, provide powerful tools that increase developer efficiency and reduce development time

L

E

I

M

D

Enhanced Performance

Includes performance improvements and optimizations that lead to better resource utilization and faster application execution

Modern Language Features

Features like sealed classes and pattern matching for switch enhance code readability, expressiveness, and maintainability

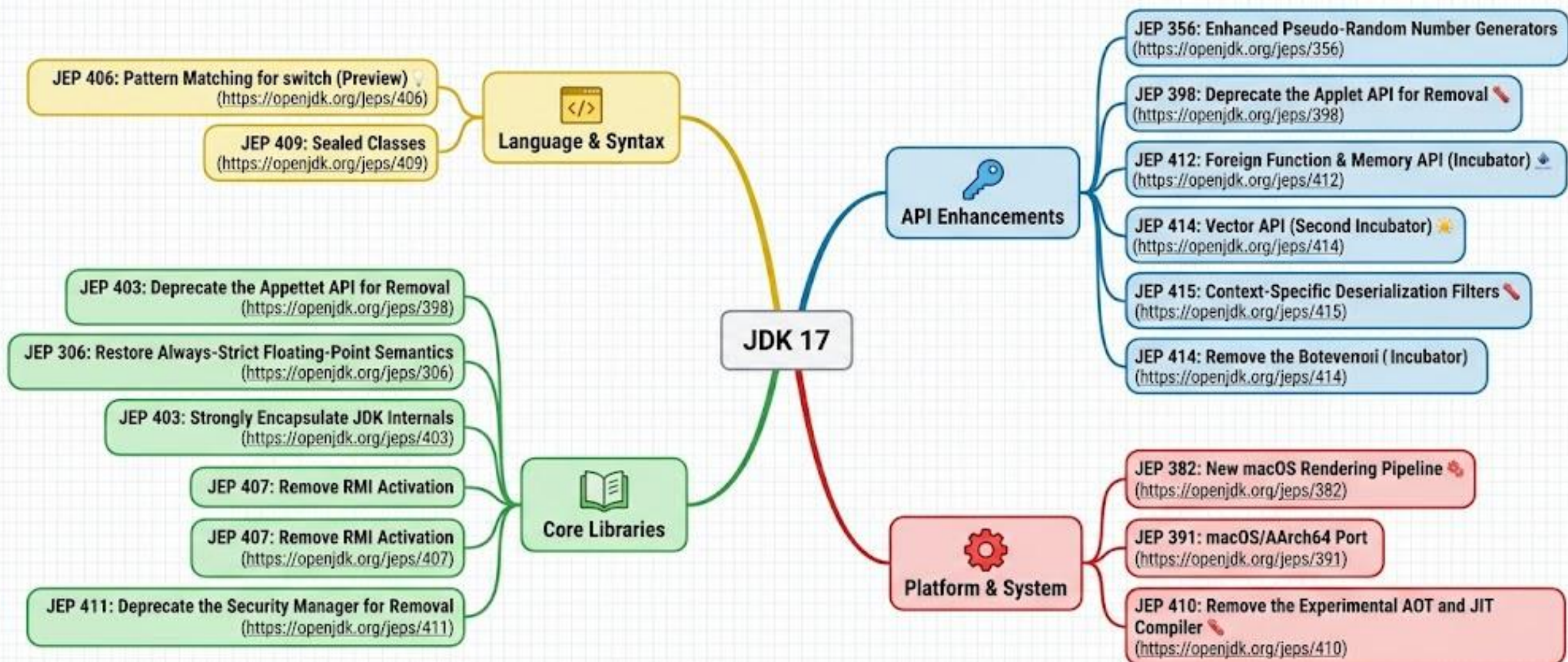
Key New Features in Java 17

1. Pattern Matching for switch (Preview)
2. Sealed Classes
3. Enhanced Pseudo-Random Number Generators
4. New Applet API Removal
5. Context-Specific Deserialization Filters
6. Foreign Function & Memory API (Incubator)
7. Vector API (Second Incubator)
8. Strong Encapsulation of JDK Internals by Default
9. Deprecate and Disable Biased Locking
10. macOS/AArch64 Port

- A **preview feature** is a new feature of the Java that is fully specified, fully implemented, and yet impermanent. It is available in a JDK feature release to provoke developer feedback based on real world use; this may lead to it becoming permanent in a future Java SE Platform.
- A **final feature** is a new feature of the Java that is fully specified, fully implemented, and permanent.
- **Incubator** modules are a means of putting non-final APIs and non-final tools in the hands of developers, while the APIs/tools progress towards either finalization or removal in a future release.

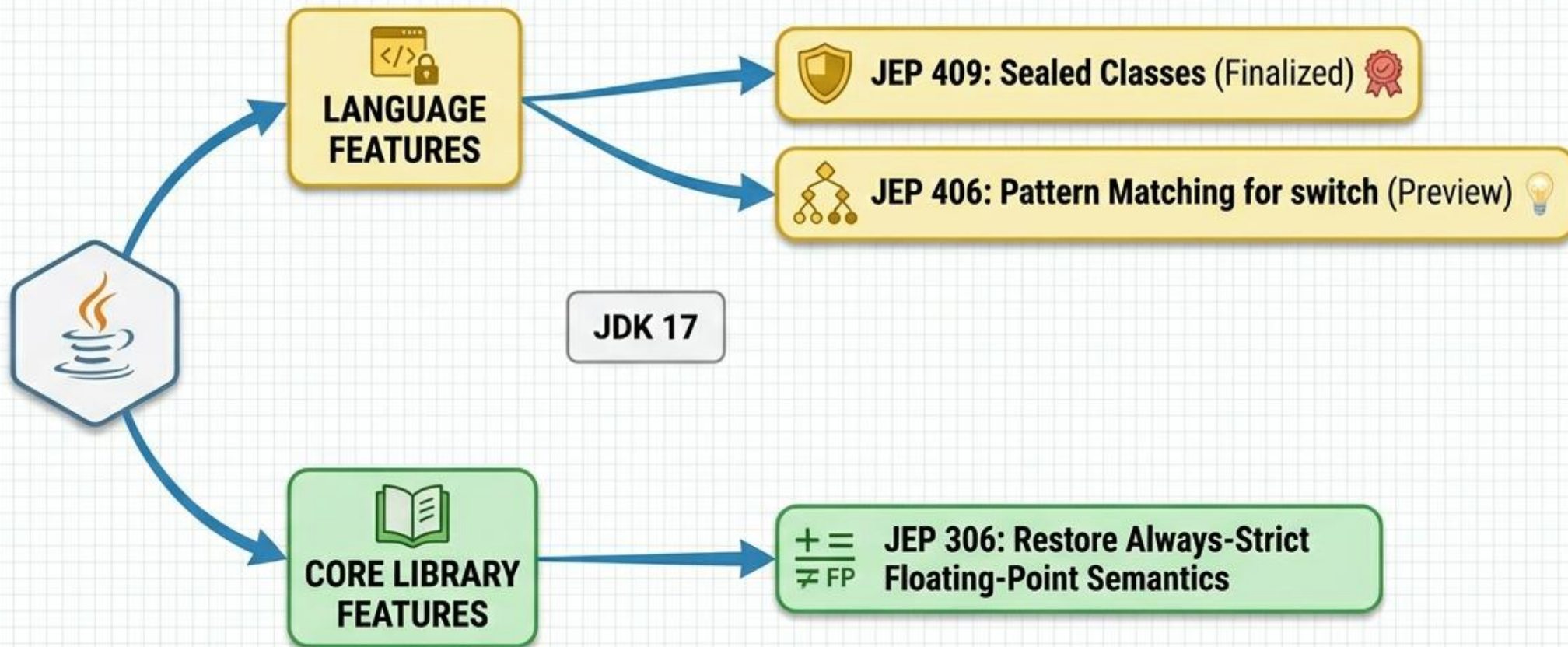
JDK 17 FEATURES (14 JEPs INTEGRATED)

As of 2021/09/13 23:38 UTC



JDK 17 KEY FEATURES CLASSIFICATION

Based on JDK 17 Features Page, with status as of 2021/09/13 23:38 UTC



Features in Java 17 (Sealed Classes)

Sealed Classes

Sealed classes and interfaces are a powerful feature introduced in Java 17. They allow you to restrict which classes or interfaces can extend or implement them. This feature provides a way to create a more controlled and predictable class hierarchy.

- Restrict which other classes or interfaces may extend or implement them
- Enhance the modeling of class hierarchies and provide better control over inheritance

Benefits:

- Ensures a limited set of subclasses, which can be easier to manage and understand
- Improves code readability and maintainability by explicitly specifying permitted subclasses
- Enables more secure and reliable designs by preventing unintended extensions

Features in Java 17 (Sealed Classes)

Implementation of Sealed Classes

```
1  // Sealed class BankAccount
2  public sealed class BankAccount permits CheckingAccount, SavingsAccount, InvestmentAccount {
3      // Define common methods or properties here if needed
4  }
5  // Final subclass CheckingAccount
6  public final class CheckingAccount extends BankAccount { 1 usage
7      // Additional methods or properties specific to CheckingAccount if needed
8  }
9  // Final subclass SavingsAccount
10 public final class SavingsAccount extends BankAccount { 1 usage
11     // Additional methods or properties specific to SavingsAccount if needed
12 }
13 // Final subclass InvestmentAccount
14 public final class InvestmentAccount extends BankAccount { 1 usage
15     // Additional methods or properties specific to InvestmentAccount if needed
16 }
```

Use Cases:

- Defining a fixed set of types for a particular domain (e.g., different kinds of shapes, commands, etc.)
- Enhancing security by limiting which classes can extend a base class
- Providing a clear and enforced API for developers using the class

Features in Java 17 (Pattern Matching for switch [Preview])

Pattern Matching for Switch (Preview)

Pattern Matching for switch is an enhancement in Java 17 that allows you to use patterns directly in switch statements and expressions. This feature simplifies complex conditional logic and enhances code readability and maintainability.

- Simplifies the syntax of complex switch statements
- Allows patterns to be used directly within switch statements and expressions

Benefits:

- More readable and concise code
- Enhanced type safety by ensuring that all cases are covered
- Reduces boilerplate code associated with type checks and casts

Features in Java 17 (Pattern Matching for switch [Preview])

Implementation of Pattern Matching for Switch (Preview)

```
1 public class Demo { no usages
2     public static void main(String[] args) {
3         Object obj = getSampleObject(); // Example object to test the switch
4         switch (obj) {
5             case Integer i -> System.out.println("Integer: " + i);
6             case String s -> System.out.println("String: " + s);
7             default -> throw new IllegalStateException("Unexpected value: " + obj);
8         }
9     }
10    private static Object getSampleObject() { 1 usage
11        // Return different types for testing
12        // You can change this to test different types
13        return 42; // Example: returning an Integer
14    }
15 }
```

Use Cases:

- Handling different types of objects in a collection
- Simplifying the processing logic based on the type of input
- Improving the clarity of conditional logic by reducing nested if-else statements

Restore Always-Strict Floating-Point Semantics

- Java 17 made floating-point predictable everywhere by making **strict the default**
- Goal - **consistent precision**
 - not performance/high precision
- JEP 306 in Java 17 removed the distinction between strict and default floating-point behavior by making all floating-point operations strictly follow IEEE 754 semantics.
- Earlier, Java allowed extended precision on some hardware, leading to inconsistent results.
- Now, all computations are deterministic and strictfp is effectively redundant.

From Java 17 onward, all floating-point operations behave as if **strictfp** is applied everywhere by default.

Before Java 17

```
▪ public class Test {  
    public static void main(String[] args) {  
        double a = 1e16;  
        double b = 1.0;  
  
        double result = (a + b) - a;  
        System.out.println(result);  
    }  
}
```

Output could be:

1.0 OR 0.0 (depending on hardware)

-> Extra precision during intermediate calculation

Java 17 (After JEP 306)

```
▪ public class Test {  
    public static void main(String[] args) {  
        double a = 1e16;  
        double b = 1.0;  
  
        double result = (a + b) - a;  
        System.out.println(result);  
    }  
}
```

Output is ALWAYS:

0.0

- ✓ Deterministic
- ✓ Predictable
- ✓ Platform-independent

. Core Libraries and APIs

JEP 356:
Enhanced
Pseudo-Random
Number
Generators



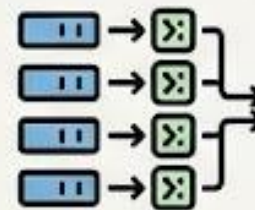
JEP 415:
Context-Specific
Deserialization
Filters



JEP 412:
Foreign Function
& Memory API
(Incubator)



JEP 414:
Vector API
(Second
Incubator)



Features in Java 17 (Enhanced Pseudo-Random Number Generators)

API enhancements

Enhanced Pseudo-Random Number Generators

Java 17 introduces several enhancements and features related to random number generation, building upon the existing *java.util.Random* and *java.util.concurrent.ThreadLocalRandom* classes. Here are some of the key enhancements related to pseudo-random number generation in Java 17:

- **New Interface: RandomGenerator** Abstraction for different random number generators.
- **SplittableRandom Implementation** Efficient generation of random numbers across threads. It Supports various algorithms and distributions.
- **Enhanced SecureRandom** Supports `ThreadLocalRandom.current()` for concurrency.
- **Cryptographically** strong randomness seeding.
- **Control and Flexibility APIs** Manage and configure random generators.
- **Performance Improvements** Enhanced algorithms for faster random number generation.

Features in Java 17 (Enhanced Pseudo-Random Number Generators)

Implementation of Enhanced Pseudo-Random Number Generators

```
1  import java.util.random.RandomGenerator;  
2  import java.util.random.RandomGeneratorFactory;  
3  
4  public class RandomGeneratorExample { no usages  
5      public static void main(String[] args) {  
6          // Create a RandomGenerator using RandomGeneratorFactory  
7          RandomGenerator random = RandomGeneratorFactory.of("Xoshiro256PlusPlus").create();  
8  
9          // Generate and print a random integer  
10         System.out.println("Random Integer: " + random.nextInt());  
11  
12         // Generate and print a random double  
13         System.out.println("Random Double: " + random.nextDouble());  
14     }  
15 }
```

Use Cases:

- **Simulation:** Accurate modeling of complex systems.
- **Security:** Cryptographic operations and secure communication protocols.
- **Gaming:** Random event generation in gaming applications.
- **Analytics:** Statistical analysis and data sampling.

Features in Java 17 (Context-Specific Deserialization Filters)

Context-Specific Deserialization Filters

Context-Specific Deserialization Filters provide a mechanism to control and restrict the deserialization process of objects based on contextual information. This is particularly useful in scenarios where deserialization of arbitrary objects from untrusted sources could pose security risks.

Key Features:

- **Filter Interface:** *java.io.ObjectInputFilter* provides methods to control deserialization behavior.
- **Contextual Criteria:** Define filters based on caller class, code source, or specific attributes.
- **Granular Control:** Allows per-call, per-thread, or per-application filters.

Benefits:

- **Security Enhancement:** Prevents deserialization attacks by limiting allowed object types.
- **Flexibility:** Customize filters based on specific application requirements.
- **Compatibility:** Integrates with existing deserialization mechanisms in Java.

Features in Java 17 (Context-Specific Deserialization Filters)

Implementation of Context-Specific Deserialization Filters

```
ObjectInputFilter filter = ObjectInputFilter.Config.createFilter("MyFilter", filterInfo -> {  
    // Implement filter logic based on filterInfo  
    // Example: Allow all objects to be deserialized  
    return ObjectInputFilter.Status.ALLOWED;  
});
```

Use Cases:

- Network Communication:** Apply stricter filters for deserialization over network protocols.
- Sensitive Data Handling:** Control deserialization in environments handling sensitive information.
- Third-Party Libraries:** Protect against vulnerabilities in third-party library deserialization.

Features in Java 17 (Foreign Function & Memory API (Incubator), Vector API (Second Incubator))

Foreign Function & Memory API (Incubator)

Facilitates seamless interaction with native code and efficient management of native memory directly from Java.

Key Features

- **Native Library Interaction:** Enables calling native functions directly from Java without JNI (Java Native Interface) overhead.
- **Memory Access:** Provides safe and efficient access to native memory regions from Java code.
- **Type Safety:** Ensures type safety when interacting with native data structures.

Vector API (Second Incubator)

The Vector API (Second Incubator) in Java 17 introduces a set of features aimed at enabling efficient SIMD (Single Instruction, Multiple Data) programming in Java

Key Features

- **SIMD Parallelism:** Enables efficient parallel processing of data using vector operations.
- **Performance:** Enhances computational throughput for numerical and data-intensive tasks.
- **Platform Support:** Designed to leverage hardware SIMD support for optimized execution.

JAVA 17: PLATFORM ENHANCEMENTS

JVM and Internals (Security & Performance)

JEP 403: Strongly Encapsulate
JDK Internals



SECURED JDK



PREVENTS DIRECT
INTERNAL API ACCESS



`sun.misc.Unsafe`

IMPROVED SECURITY AND STABILITY

Ports & Graphics (Platform Specific)

macOS/AArch64 Port (JEP 391)



NATIVE ARM64
(M1/M2)
PERFORMANCE



New macOS Rendering Pipeline (JEP 382)



MODERN, SMOOTH
GRAPHICS PERFORMANCE

LONG-TERM
COMPATIBILITY

Features in Java 17 (Strong Encapsulation of JDK Internals by Default)

Strong Encapsulation of JDK Internals by Default

In Java 17, a significant enhancement was introduced regarding the strong encapsulation of JDK internals by default. This change aims to improve the security and maintainability of Java applications by restricting direct access to internal APIs that are not part of the public Java API specification.

- **Access Restriction:** Access to internal JDK classes and APIs (those in packages like *sun.**, *com.sun.**, *jdk.internal.**, etc.) is restricted by default. Attempting to access these APIs from code outside the JDK itself will result in compilation errors or runtime exceptions.
- **Impact on Developers:**
 - **Migration:** Developers are required to migrate their code away from using internal APIs to public APIs or supported alternatives.
 - **Security:** This change enhances the security of Java applications by reducing the attack surface exposed through potentially insecure internal APIs.
 - **Maintainability:** Encourages developers to write more robust and maintainable code by relying only on stable, documented public APIs.
- **Migration Strategies:**
 - **Use Public APIs:** Identify and use public APIs or alternative libraries that provide equivalent functionality.
 - **Module System:** Utilize Java's module system (introduced in Java 9) to encapsulate internal implementations and expose only necessary public interfaces.
 - **Compatibility:** Use tools like the `jdeps` tool to analyze dependencies and identify usage of internal APIs that need migration.

Features in Java 17 (Strong Encapsulation of JDK Internals by Default)

Implementation of Strong Encapsulation of JDK Internals by Default

In Java 17, direct access to `sun.misc.BASE64Encoder` is restricted due to strong encapsulation of JDK internals. Therefore, you should use the `java.util.Base64` class from the standard Java API instead. Here's how you can rewrite your code using `java.util.Base64`:

```
1  //Before jdk17
2  import sun.misc.BASE64Encoder;
3
4  public class Main {
5      public static void main(String[] args) {
6          BASE64Encoder encoder = new BASE64Encoder();
7          String encoded = encoder.encode("Hello, World!".getBytes());
8          System.out.println("Encoded: " + encoded);
9      }
10 }
```

Features in Java 17 (macOS/AArch64 Port)

macOS/AArch64 Port

- **Transition to Apple Silicon**
 - **Apple Silicon:** ARM-based processors (M1, M1 Pro, M1 Max) introduced by Apple for Macs.
 - **Performance:** Improved power efficiency and performance compared to Intel x86.
- **Java 17 Support**
 - **Native Compilation:** Java 17 supports AArch64 (ARM64) architecture natively.
 - **Compatibility:** Ensures Java applications run efficiently on macOS devices with Apple Silicon.
- **Development Ecosystem**
 - **Xcode Integration:** Java development using Xcode on macOS AArch64.
 - **Universal Binaries:** Facilitates deployment of Java applications as universal binaries for both ARM and x86 architectures.
- **Benefits**
 - **Performance Boost:** Native execution on Apple Silicon enhances Java application performance.
 - **Ecosystem Integration:** Enables developers to leverage new hardware capabilities seamlessly.

JAVA 17: DEPRECATIONS & REMOVALS

DEPRECATIONS FOR REMOVAL

JEP 411: DEPRECATE THE SECURITY MANAGER



DEPRECATE
SECURITY MANAGER
REMOVAL
OLD (JAVA 1.0)
MAINTENANCE BURDEN

[link Sources]

(https://lh3.googleusercontent.com/hd-gg-dUAFIU-8Lw/MelobjuY8wUpkWWVjaqHEd10QV09gID2IQ004_1av4ey0XwamdS106rP7b2-9Lz53EEQ8wBajccos-s1024-g)

JEP 398: DEPRECATE THE APPLET API

DEPRECATE
APPLET API
REMOVAL
OBSOLETE
BROWSERS REMOVED
SUPPORT

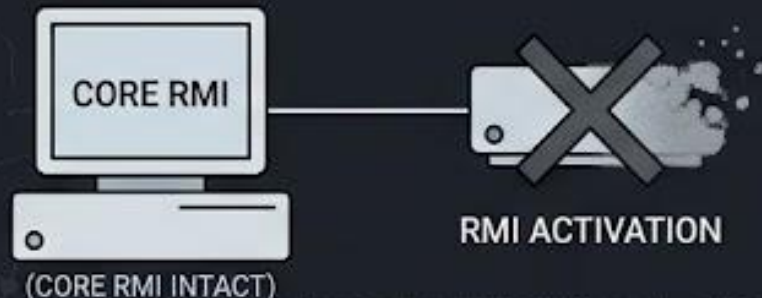


[link Sources]

(https://lh3.googleusercontent.com/hd-gg-dUAFIU-8Lw/MelobjuY8wUpkWWVjaqHEd10QV09gID2IQ004_1av4ey0XwamdS106rP7b2-9Lz53EEQ8wBajccos-s1024-g)

REMOVALS

JEP 407: REMOVE RMI ACTIVATION



REMOVE
ACTIVATION
OBSOLETE
SYSTEM

[link Sources]

(https://lh3.googleusercontent.com/hd-gg-dUAFIU-8Lw/MelobjuY8wUpkWWVjaqHEd10QV09gID2IQ004_1av4ey0XwamdS106rP7b2-9Lz53EEQ8wBajccos-s1024-g)

JEP 410: REMOVE EXPERIMENTAL AOT/JIT

REMOVE
COMPILERS
EXPERIMENTAL
GRAAL-BASED
MINIMAL USE

0110
0101
1011
0111
0110
0110
0101
0110

AOT

JIT



[link Sources]

(https://lh3.googleusercontent.com/hd-gg-dUAFIU-8Lw/MelobjuY8wUpkWWVjaqHEd10QV09gID2IQ004_1av4ey0XwamdS106rP7b2-9Lz53EEQ8wBajccos-s1024-g)

Features in Java 17 (New Applet API Removal)

Deprecate the Applet API for Removal

In JDK 17, the Applet API is marked as deprecated for removal. It means that the JDK 18 will remove the API. The Applet API removal is long overdue, considering that all browsers have either ended the support or have plans to do so. The API had already been deprecated in JDK 9, but it had not been marked for removal.

- **Applet API:** Historically used for embedding Java applets in web browsers.
- **Replacement:** HTML5, JavaScript, and modern web technologies now fulfill similar functionalities.

Reasons:

- Security vulnerabilities
- Decreased usage
- Modern web standards led to this decision

Implications:

- **End of Life:** Applets are no longer supported for running in web browsers as of Java 11 and beyond.
- **Transition:** Developers and organizations must migrate existing applet-based applications to alternative technologies.

Frequently asked Interview Questions

Does Sealed classes Break OCP?

- sealed interface Customer permits RegularCustomer, PremiumCustomer {}
- final class RegularCustomer implements Customer {}
- final class PremiumCustomer implements Customer {}
- class DiscountService {
 public double getDiscount(Customer customer) {
 return switch (customer) {
 case RegularCustomer rc -> 10.0;
 case PremiumCustomer pc -> 20.0;
 };
 }
}
- *Add a new customer type: GoldCustomer with 30% discount”*
- *final class GoldCustomer implements Customer {}*

Which principle is broken here?

```
sealed class Bird permits Sparrow, Penguin {  
    abstract void fly();  
}  
  
final class Sparrow extends Bird {  
    void fly() {  
        System.out.println("Flying");  
    }  
}  
  
final class Penguin extends Bird {  
    void fly() {  
        throw new UnsupportedOperationException("Cannot fly");  
    }  
}
```

Fix and Better approach :

```
sealed interface Bird permits FlyingBird, NonFlyingBird {}
```

```
sealed interface FlyingBird extends Bird permits Sparrow {  
    void fly();  
}
```

```
sealed interface NonFlyingBird extends Bird permits Penguin {}
```

```
final class Sparrow implements FlyingBird {  
    public void fly() {  
        System.out.println("Flying");  
    }  
}
```

```
final class Penguin implements NonFlyingBird {}
```

Question:

Difference between Enum and sealed classes

Question:

What happens if you miss a case in sealed + switch?

Question:

You're designing a payment system with limited extension points. Would you use sealed classes or interfaces? Why?

How would sealed classes behave if your system is split across multiple modules?

Can a sealed class be extended dynamically at runtime (e.g., via reflection or proxies)?

What happens if a permitted subclass is missing at runtime?

Question:

Replace a chain of if-else conditions with switch expressions and evaluate trade-offs.

Question:

1. can records have methods?
2. Use cases and drawbacks of records?
3. Records vs Lombok
4. can we extend a record
5. can record implement a interface?
6. Are records truly immutable?

Question:

What features help reduce boilerplate?

Records

Pattern matching

Text blocks

Question: Why should we move from Java 8 to 17?

Question: what are GC improvements in java 17?

- **G1 GC is still the default**
 - Garbage-First Garbage Collector (G1) remains the default collector.
 - Improved performance, lower pause times compared to older versions.
- **ZGC is production-ready**
 - Z Garbage Collector is no longer experimental.
 - Designed for **very low latency (sub-10ms pauses)**.
 - Now supports **large heaps (multi-terabyte)** and better stability.
- **Shenandoah improvements**
 - Shenandoah Garbage Collector also improved (low pause GC).
 - Better performance and reduced overhead.
- **Parallel GC enhancements**
 - Parallel Garbage Collector got tuning improvements for throughput.
- **Strong encapsulation impact**
 - Internal GC tuning APIs are more restricted (due to stronger encapsulation in Java 17).

- ◆ **Removed / Deprecated**

No major GC removed in Java 17, but older ones like CMS were already removed earlier (Java 14).

You have a low-latency trading system. Which GC would you choose in Java 17 and why?

Migration Scenarios (VERY COMMON)

You are migrating from Java 8 to Java 17. What breaking changes do you expect?

How would you incrementally migrate a large monolith to Java 17?

Thank you